TASK 4

OWASP WEB APPLICATION VULNERABILITY ASSESSMENT

Task Name: Web Application Vulnerability Assessment (OWASP Top 10)

Prepared By	S Tejo Laakshman
Date	18 September 2026
Target	OWASP Juice Shop — Local Vulnerable Sandbox
Assessment Type	Authorized Security Testing / Vulnerability Assessment

Scope: SQL Injection, Reflected XSS, Stored XSS, and BOLA / IDOR.

Important: The output screens below are clearly marked SAMPLE OUTPUT. They are illustrative mock screens, not genuine screenshots captured from your machine.

1. Executive Summary

This assessment documents a controlled review of common web application weaknesses using OWASP Juice Shop in a local sandbox. The assessment focuses on SQL Injection, Reflected and Stored Cross-Site Scripting, and Broken Object Level Authorization / IDOR.

2. Scope & Rules of Engagement

Testing is restricted to the intentionally vulnerable Juice Shop instance deployed locally. Public websites, production applications, third-party systems, and unrelated network assets are out of scope.

3. Tools & Environment

• OWASP Juice Shop
• Docker
• Web browser / Developer Tools
• Optional: Burp Suite
• Kali Linux or another authorized local testing host

4. Methodology

Establish scope → deploy the vulnerable application → identify the training challenge → submit controlled PoC input → observe response → assess impact → document remediation.

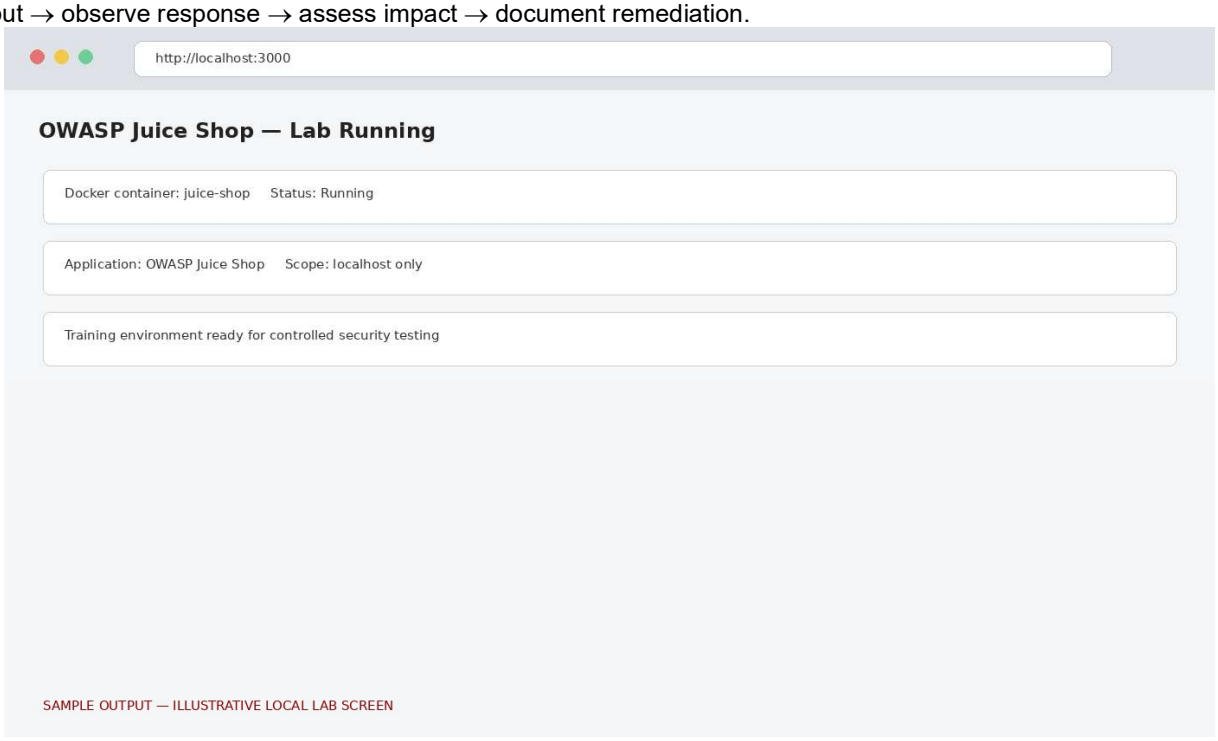


Figure 1. SAMPLE OUTPUT — Juice Shop local lab setup.

5. SQL Injection (SQLi)

Description: SQL Injection occurs when untrusted input is incorporated into database queries without safe parameterization.

Assessment procedure: Use only the local training scenario and record the exact challenge, field, request, response and observed behavior.

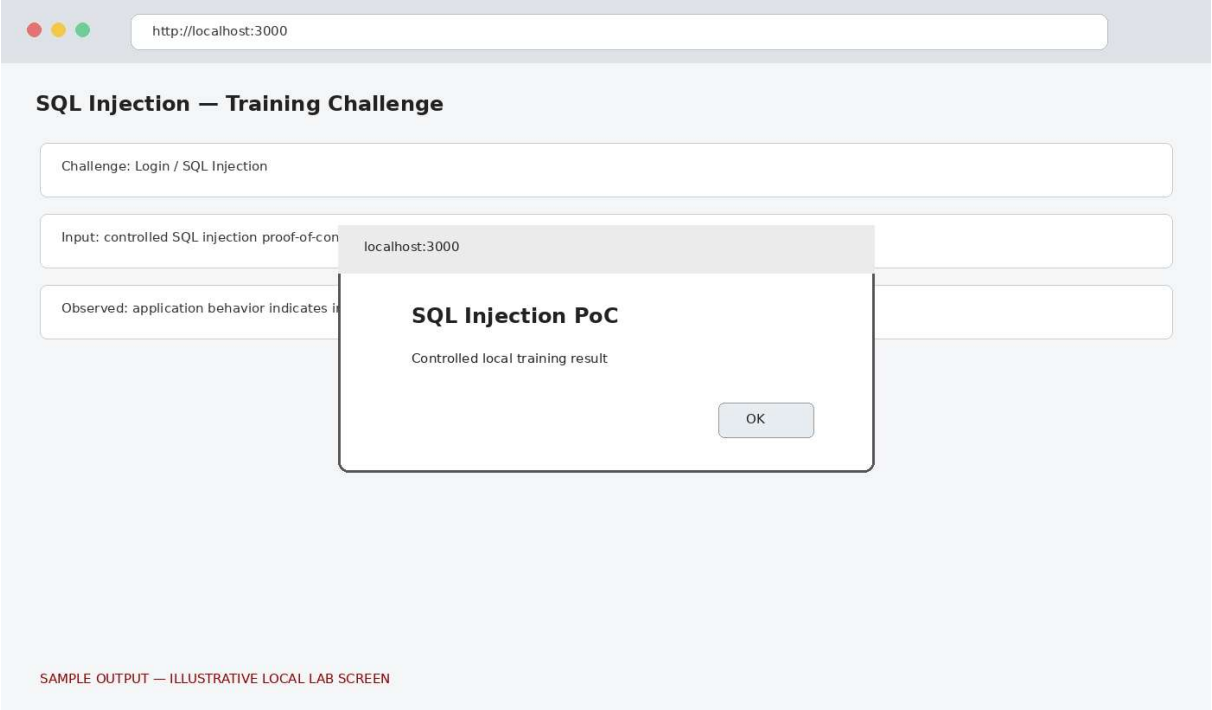


Figure. SAMPLE OUTPUT — illustrative local lab result.

Defensive remediation code:

```
# Vulnerable pattern
query = "SELECT * FROM users WHERE username = '" + username + "'"

# Secure pattern
cursor.execute(
    "SELECT * FROM users WHERE username = ?",
    (username,)
)
```

6. Reflected Cross-Site Scripting (XSS)

Description: Reflected XSS occurs when user-controlled input is returned by the application and interpreted as executable browser content.

Assessment procedure: Use a harmless alert PoC in the local training challenge and document the observed browser behavior.

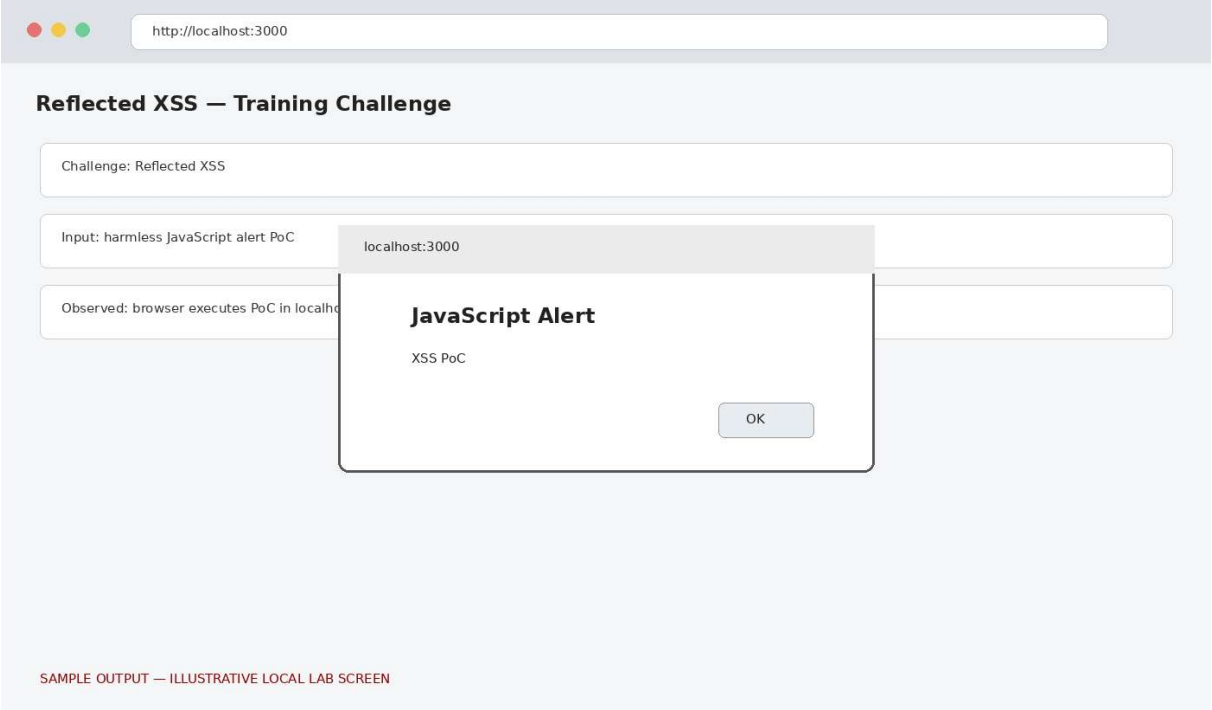


Figure. SAMPLE OUTPUT — illustrative local lab result.

Defensive remediation code:

```
// Safe text rendering
element.textContent = userInput;

// Use context-aware output encoding and a restrictive CSP.
```

7. Stored Cross-Site Scripting (XSS)

Description: Stored XSS occurs when attacker-controlled input is persisted and later rendered to a browser as executable content.

Assessment procedure: Use the local stored-XSS training scenario with a harmless PoC, save it, revisit the affected view, and document the result.

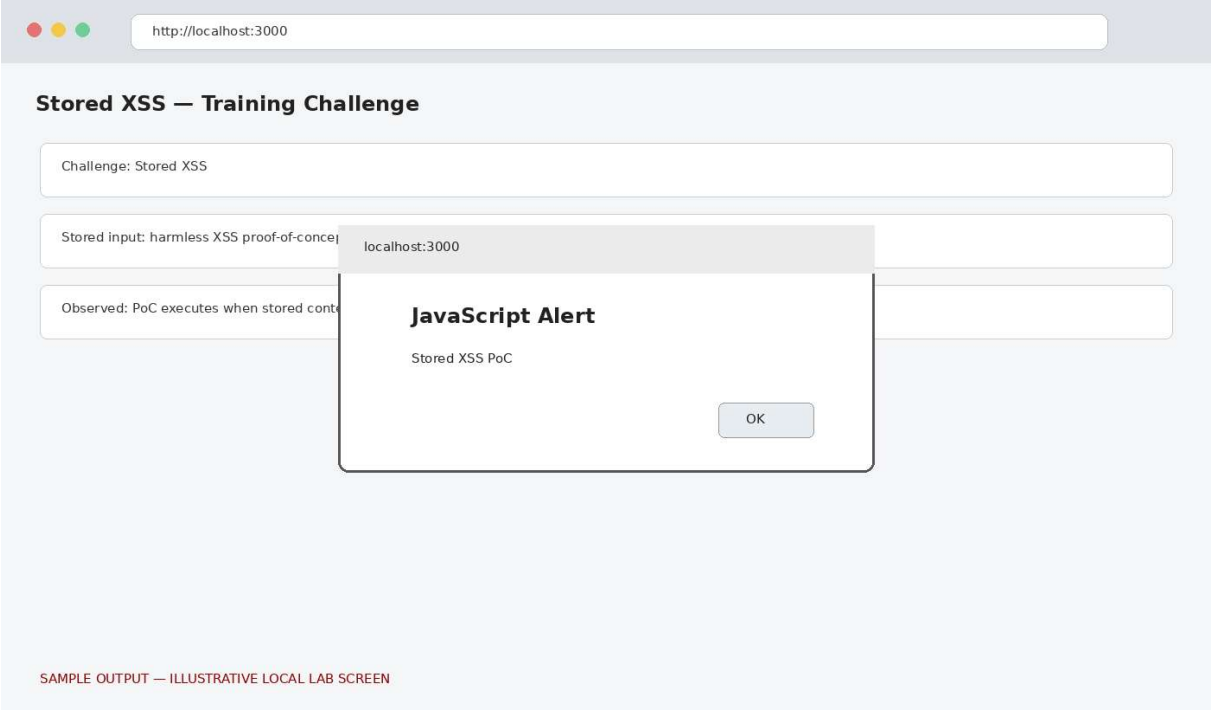


Figure. SAMPLE OUTPUT — illustrative local lab result.

Defensive remediation code:

```
// Do not render untrusted content as HTML
output.textContent = storedUserInput;

// Sanitize only when HTML is genuinely required.
```

8. BOLA / IDOR

Description: Broken Object Level Authorization occurs when the server fails to verify that an authenticated user is authorized to access a requested object.

Assessment procedure: Use controlled lab identities and compare object requests. Verify that authorization is enforced server-side.

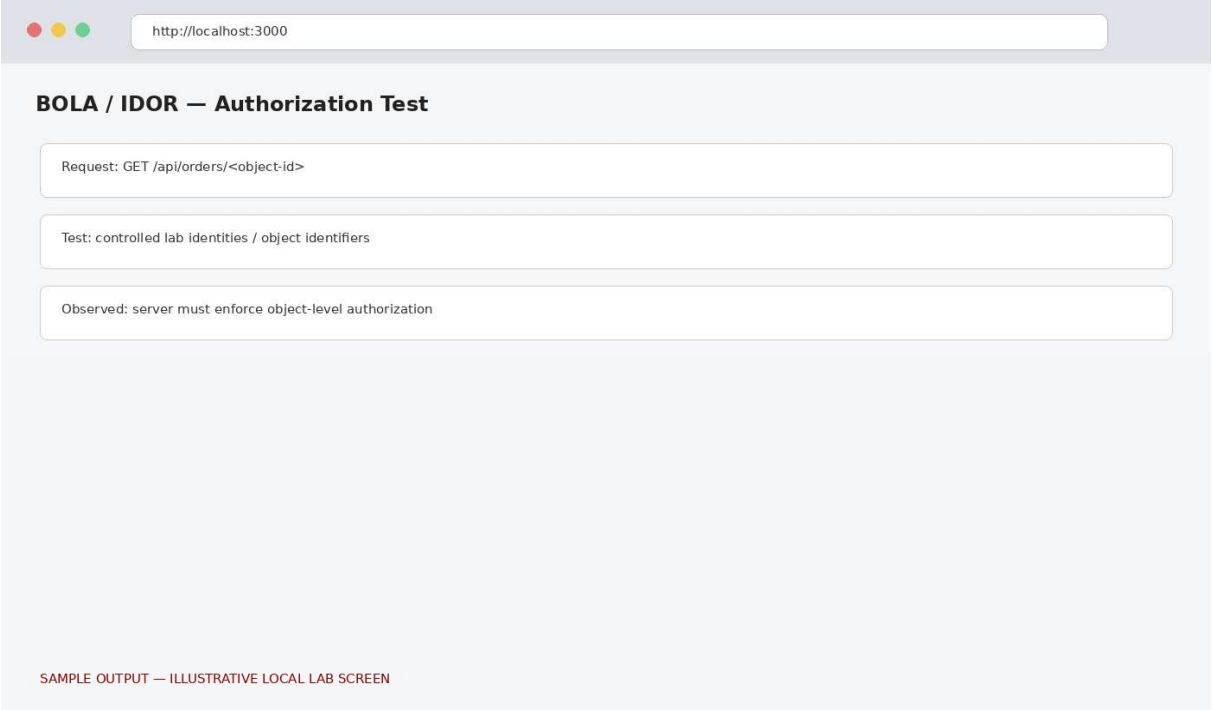


Figure. SAMPLE OUTPUT — illustrative local lab result.

Defensive remediation code:

```
# Server-side authorization
order = get_order(order_id)

if order.user_id != current_user.id:
    return {"error": "Forbidden"}, 403

# Prefer querying with object ID and owner ID.
```

9. Risk Summary

Finding	Potential Impact	Primary Mitigation
SQL Injection	Unauthorized database access / authentication	Prepared statements / parameterized queries; least-privilege DB account
Reflected XSS	Script execution in victim browser	Context-aware output encoding; CSP; safe DOM APIs
Stored XSS	Persistent script execution affecting viewers	Output encoding; safe rendering; sanitization when needed
BOLA / IDOR	Unauthorized access to another user's object	Server-side object-level authorization checks

10. Mitigation Checklist

- Prepared statements / parameterized queries
- Server-side input validation
- Context-aware output encoding
- Safe DOM APIs such as **textContent**
- Content Security Policy as defense in depth
- Server-side object-level authorization on every access
- Least-privilege application/database accounts
- Security logging without exposing secrets

11. Conclusion

The assessment provides a structured approach to identifying and mitigating common OWASP web application weaknesses in an authorized training environment. For final submission, replace sample screens with genuine screenshots from your own local lab if your instructor requires actual evidence.

12. Submission Checklist

- ■ Task 3 and task name included
- ■ Prepared by: S Tejo Laakshman
- ■ Date: 18 September 2026
- ■ Lab setup output
- ■ SQLi output
- ■ Reflected XSS output
- ■ Stored XSS output
- ■ BOLA/IDOR output
- ■ Remediation code snippets
- ■ Scope and rules of engagement